TECHNICAL PLAN · CONFIDENTIAL

Present

Backend Development Plan — from mobile demo app to a production-ready school attendance platform.

Scope: Full review of current app features (Expo / React Native, local-only storage) and a target backend architecture plan.

Prepared for: Present project team

Version: 1.0

1. Executive Summary

Present is currently an Expo / React Native mobile app that is **entirely client-side**: all data (teachers, classes, students, attendance sessions) is either hard-coded in source (`data/*.ts`) or persisted locally on-device via `AsyncStorage` (through `zustand/persist`). There is currently **no server, database, API, or third-party service** — QR scanning, parent notifications, and authentication are all simulated.

This document inventories every feature in the app, identifies what is currently "faked," and proposes a target backend architecture, data schema, API endpoint list, a notification strategy (WhatsApp), an offline sync strategy, and a phased implementation roadmap.

> **Key finding:** the current local data model only allows one checked-in class per day, total (across all classes). This is a demo-time simplification and will need to be redesigned in the backend model to support multiple sessions (class + subject) per day per teacher.

2. Table of Contents

1. 1. Executive Summary
2. 2. Table of Contents
3. 3. Feature Inventory & Backend Gaps
4. 4. Proposed Target Architecture
5. 5. Data Model (Schema)
6. 6. API Endpoints
7. 7. QR Code Strategy
8. 8. Parent Notifications (WhatsApp)
9. 9. File Storage (Student Photos)
10. 10. Authentication & Security
11. 11. Offline / Sync Strategy
12. 12. Multi-School & Scalability
13. 13. Migration Plan From Current Data
14. 14. Phased Roadmap
15. 15. Open Questions / Decisions Needed

3. Feature Inventory & Backend Gaps

Every screen in the app was reviewed to determine what it does today (client-only) and what a real backend needs to provide.

Feature	Current Behavior (Simulated)	Backend Need
Teacher Login `LOGIN.TSX`	The typed email is matched against a static list of 5 teachers; no password is actually verified.	Real authentication (email + password or magic link), JWT tokens, session management.
Teacher Profile `PROFILE.TSX`	Stats (sessions checked in, classes covered) computed from the local sessions array.	Server-side aggregated stats endpoint per teacher.
Subject Assignment `DATA/TEACHERS.TS`	Each teacher has 1–N hard-coded subjects; one subject selected once and applied for the day.	Teacher↔Subject join table, manageable by a school administrator.
Select / Scan a Class `HOME.TSX, CLASS-SCANNER.TSX`	"Scanning" picks a random class from a static list after a 2.5s timer.	Real classes in the database, real QR decoding, rotating class tokens.
Student ID Card / QR GAP `ID-CARD.TSX`	Displayed QR is a fake static grid; the student ID is generated client-side (`SC-2024-88xx`).	Durable unique student IDs, signed QR (anti card-duplication), QR-resolution endpoint.
Individual Student Scan `SCANNER.TSX`	Picks a random student, simulates a 15% error rate, checks duplicates locally.	Real QR decoding, student lookup, idempotent check-in with server timestamp.
Attendance Register `REGISTER.TSX`	Present / Not Yet Arrived tabs, manual marking, a search field that isn't wired up.	Attendance CRUD, server-side student search endpoint.
Explicit "Absent" Marking `STORE.MARKSTUDENTABSENT`	Distinguishes "unaccounted for" from "marked absent" via a local boolean flag.	Attendance status stored in DB: `pending | present | absent`.
Student Registration CRITICAL GAP `REGISTER-STUDENT.TSX`	The "Save" button just runs `router.back()` — **nothing is actually persisted**, including the photo.	Student creation endpoint, photo upload, guardian WhatsApp contact storage.
Scan Result Screens `SCAN-RESULT/*.TSX`	The success screen shows "Parent notified" without sending anything.	Real notification dispatch (WhatsApp/SMS) with delivery status.
History `HISTORY.TSX`	Lists locally stored sessions; only one session possible per day, total.	Full history per school/teacher/class, pagination, date filtering.
Data Persistence CRITICAL GAP	`AsyncStorage` only — data lost on uninstall, not shared across devices.	Server database, account tied to teacher, multi-device sync.

4. Proposed Target Architecture

Two options are viable depending on budget, timeline, and team size:

Option A — Backend-as-a-Service (recommended for MVP)

Supabase (managed Postgres + Auth + Storage + Realtime + Row Level Security) makes it possible to ship a working backend in a few weeks without managing server infrastructure.

- **Auth**: Supabase Auth (email/password, session management, JWT).
- **Database**: managed Postgres with Row Level Security (per-school data isolation).
- **Storage**: Supabase Storage for student photos.
- **Realtime**: live subscription to attendance updates (useful for an admin dashboard).
- **Functions**: Edge Functions for business logic (QR validation, triggering WhatsApp notifications).

Option B — Custom Backend

Recommended if the app needs to grow into a large-scale, multi-school SaaS product.

- **API**: Node.js + NestJS (or Fastify), modular architecture by domain (auth, schools, attendance).
- **Database**: PostgreSQL (via Prisma or TypeORM).
- **Queue**: Redis + BullMQ for async jobs (notifications).
- **Object storage**: S3 (or compatible) for photos.
- **Hosting**: containers (Docker) on a cloud provider, with attention to data residency for DRC/Africa if required.

> **Recommendation:** start with Option A (Supabase) to validate the product quickly, then migrate to Option B only if customization or scale requirements justify it. The proposed data schema (Section 5) is designed to work with either option.

5. Data Model (Schema)

The schema includes a `school_id` on every table from the start to support a multi-school model, even if the initial launch covers only one school.

schools

Column	Type	Description
id	uuid, PK	School identifier
name	text	School name
country	text	E.g. "CD" (DR Congo)
timezone	text	E.g. "Africa/Kinshasa"
created_at	timestamptz	

teachers

Column	Type	Description
id	uuid, PK	
school_id	uuid, FK	
full_name	text	
email	text, unique	Used for login
password_hash	text	Managed by the auth provider
avatar_url	text	
role	enum	`teacher | admin`

subjects & teacher_subjects

Table	Columns	Description
subjects	id, school_id, code, label	E.g. `MATHS / Mathématiques`
teacher_subjects	teacher_id, subject_id	N↔N relationship

classes

Column	Type	Description
id	uuid, PK	
school_id	uuid, FK	
name	text	E.g. "Grade 5B"

qr_token	text, unique	Rotating token encoded in the class QR

students

Column	Type	Description
id	uuid, PK	
class_id	uuid, FK	
full_name	text	
student_code	text, unique	Human-readable ID printed on the card
qr_token	text, unique	Signed token encoded in the student QR
photo_url	text	
guardian_name	text	
guardian_whatsapp	text	With country code

sessions

Column	Type	Description
id	uuid, PK	
class_id	uuid, FK	
teacher_id	uuid, FK	
subject_id	uuid, FK	
date	date	
started_at	timestamptz	

Recommended uniqueness constraint: (class_id, subject_id, date) — replaces today's "one session per day, total" limitation with a correct per-class, per-subject constraint.

attendance_records

Column	Type	Description
id	uuid, PK	
session_id	uuid, FK	
student_id	uuid, FK	
status	enum	`pending | present | absent`
marked_at	timestamptz	
method	enum	`qr_scan | manual`

notifications

Column	Type	Description
id	uuid, PK	
attendance_record_id	uuid, FK	
channel	enum	`whatsapp | sms`
status	enum	`queued | sent | delivered | failed`
sent_at	timestamptz	

6. API Endpoints

Overview of REST endpoints needed, grouped by functional domain.

Auth

Method	Route	Description
POST	`/auth/login`	Teacher login (email + password)
POST	`/auth/refresh`	Refresh access token
POST	`/auth/logout`	Invalidate session

Teachers & Subjects

Method	Route	Description
GET	`/teachers/me`	Logged-in teacher's profile
GET	`/teachers/me/subjects`	Assigned subjects
GET	`/teachers/me/stats`	Sessions / classes covered

Classes & Students

Method	Route	Description
GET	`/classes`	List the school's classes
GET	`/classes/resolve-qr`	Resolve a scanned class QR token
GET	`/classes/:id/roster`	List a class's students
POST	`/students`	Register a new student (multipart, photo included)
GET	`/students/:id`	Student detail (for the ID card)
GET	`/students/resolve-qr`	Resolve a scanned student QR token

Sessions & Attendance

Method	Route	Description
POST	`/sessions`	Open a session (class + subject + date)
GET	`/sessions/today`	Today's sessions for the logged-in teacher
POST	`/sessions/:id/attendance/scan`	Record attendance via student QR
POST	`/sessions/:id/attendance/:studentId/present`	Manually mark present
POST	`/sessions/:id/attendance/:studentId/absent`	Manually mark absent
GET	`/sessions/:id/attendance?search=`	List + search attendance

History

Method	Route	Description
GET	`/history?teacherId=&from=&to=`	Paginated session history
GET	`/history/:sessionId`	Detail of a past session

7. QR Code Strategy

Class QR

Encodes `{ schoolId, classId, rotatingToken }`. The rotating token is regenerated periodically (e.g. each term) so classroom posters can be reprinted without compromising security if an image is shared.

Student QR (ID Card)

Encodes `{ studentId, signature }`, where `signature` is an HMAC signed with a server-side secret. This prevents fabricating a fake card or copying a simple identifier. If a card is lost, a new token can be issued, invalidating the old one.

> The QR grid currently rendered in `components/IDCard.tsx` is purely decorative (static data) — it will need to be replaced with a real QR render (e.g. the `react-native-qrcode-svg` library) encoding the signed token returned by the API.

8. Parent Notifications (WhatsApp)

The guardian's WhatsApp number is already collected in the registration screen (`register-student.tsx`), with a country code — the natural integration target is the **WhatsApp Business Cloud API** (Meta).

1. Whenever an attendance record is set to `status = present`, a job is queued asynchronously.
2. A worker sends a message using a Meta-approved template (e.g. "{student} arrived at school at {time}").
3. Delivery status (`queued / sent / delivered / failed`) is stored and can be surfaced in the app.

> **Lead time to plan for:** Meta business verification and WhatsApp message template approval can take several weeks — start this early in the project timeline. An SMS fallback (e.g. Twilio, Africa's Talking) is recommended while approval is pending.

9. File Storage (Student Photos)

Photo selection via `expo-image-picker` already works client-side, but the image stays on-device only (local URI, never uploaded). This requires:

- An upload endpoint (multipart or pre-authorized signed URL) returning a durable `photo_url`.
- Client-side resizing/compression before upload (partially handled already via `allowsEditing` and `quality: 0.8`).
- A retention and deletion policy compliant with minors' data protection requirements.

10. Authentication & Security

- Replace the fake email-matching login with real authentication (hashed password, or magic link).
- Short-lived JWT access tokens + refresh token.
- Role-based access control (teacher vs. school administrator).
- Rate limiting on scan endpoints to prevent abuse.
- Replay protection for QR tokens (validity window, invalidation on reissue).
- Minimize data collected on minors; capture guardian consent at registration; support right to erasure.

11. Offline / Sync Strategy

Network connectivity at some schools in DRC can be intermittent. The app should keep an offline-first model:

- The local store (`zustand + AsyncStorage`) becomes a **write queue** rather than the source of truth.
- Each attendance action is created with a client-generated UUID to allow an *idempotent upsert* server-side (avoids duplicates on retry after a dropped connection).
- Background sync as soon as connectivity returns; a visible "pending sync" indicator in the UI.

12. Multi-School & Scalability

Even though the initial launch targets a single school, the proposed schema includes `school_id` on every table from day one. This avoids a costly migration later if the app is rolled out to multiple schools (SaaS model), and allows per-school data isolation via Row Level Security rules.

13. Migration Plan From Current Data

Current Source (Client)	Backend Destination
`data/classes.ts`	Seed script → `classes` + `students` tables
`data/teachers.ts`	Seed script → `teachers` + `teacher_subjects` tables
`data/subjects.ts`	Seed script → `subjects` table
`store/useAppStore.ts`	Kept as a client cache, but every action (`checkInClass` , `markStudentPresent` , `markStudentAbsent` , `setTeacher` , `setActiveSubject`) now calls the API, with optimistic UI updates.

14. Phased Roadmap

Phase 0 — Foundations

1–2 weeks

Finalize the stack choice (Option A or B), provision infrastructure, database schema and migrations, set up authentication.

Phase 1 — Core CRUD

2–3 weeks

Endpoints for schools/teachers/classes/students/subjects. Wire up `login.tsx` and `register-student.tsx` to real endpoints.

Phase 2 — Attendance Engine

2 weeks

Session + attendance endpoints, QR token issuance/validation, wire up `class-scanner.tsx`, `scanner.tsx`, `register.tsx`.

Phase 3 — Notifications

1–2 weeks

WhatsApp Business Cloud API integration + async worker, delivery status surfaced in the app.

Phase 4 — History, Stats & Offline Sync

2 weeks

History endpoints, profile stats, client-side offline queue and conflict resolution.

Phase 5 — Hardening

1–2 weeks

Security review, rate limiting, backups, observability/logging, load testing.

Total estimate: ~10–14 weeks for an MVP, with 1–2 backend engineers. Phase 3 (Meta approval) should start as early as possible, in parallel with the other phases.

15. Open Questions / Decisions Needed

- Should the app support multiple schools from launch, or a single school to start?
- Is there budget/access for Meta business verification (WhatsApp Business API)? If not, which fallback channel (SMS) should be prioritized?
- Preference between a managed solution (Supabase) to move fast, or a custom backend for more long-term control?
- Is there a data residency constraint (hosting in Africa/DRC) for regulatory or latency reasons?
- Who will handle student registration and ID card printing in real conditions — school administration, or each teacher?